

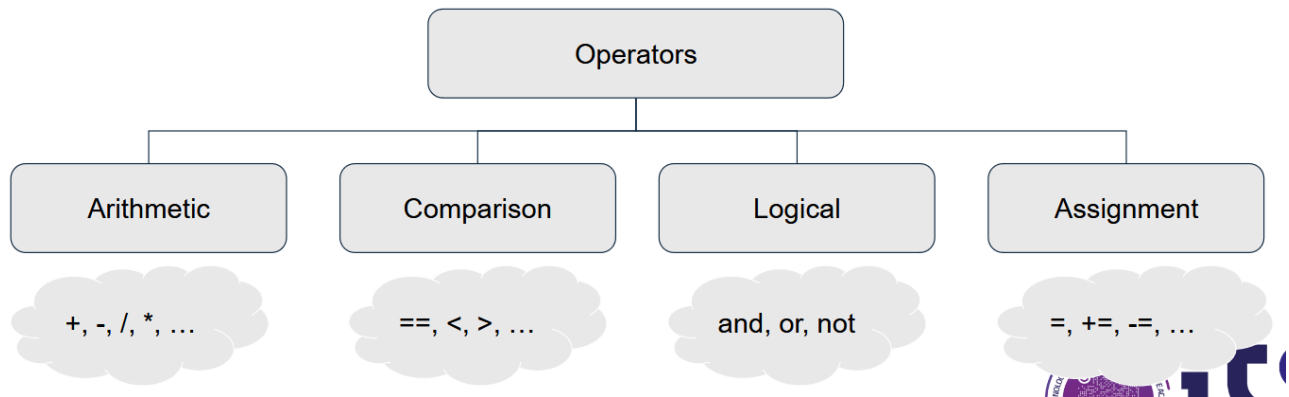
Gli Operatori

Modificare i dati in Python

Cosa sono gli operatori?

Gli operatori sono usati per eseguire operazioni su variabili e valori.

Python supporta vari tipi di operazioni, includendo operazioni aritmetiche, comparazioni, logiche e gli operatori di assegnazione.



Operatori.png

Gli operatori aritmetici

Vengono utilizzati per eseguire calcoli matematici sui numeri.

Addizione	+
Sottrazione	-
Moltiplicazione	*
Divisione	/
Potenza	()**
Divisione intera	(//)
Modulo	(%)

1. Addizione(+):

Somma dei numeri

```
Python
1 risultato = 5 + 3
```

```
2 print(risultato) # Output: 8
```

2. Sottrazione (-):

Sottrae un numero dall'altro



Python

```
1 risultato = 10 - 4
2 print(risultato) # Output: 6
```

3. Moltiplicazione(*):

Fa la moltiplicazione tra i numeri



Python

```
1 risultato = 6 * 7
2 print(risultato) # Output: 42
```

4. Divisione(/):

Fa la divisione tra i numeri e restituisce il risultato come un numero con la virgola (float)



Python

```
1 risultato = 9 / 2
2 print(risultato) # Output: 4.5
```

5. Potenza(**):

Calcola la potenza del primo numero elevato al secondo numero inserito



Python

```
1 risultato = 2 ** 3 # 2 elevato alla terza potenza
2 print(risultato) # Output: 8
```

6. Divisione intera (//):

Divisione dei numeri e restituisce solo la parte intera (integer).



Python

```
1 risultato = 9 // 2
```

```
2 print(risultato) # Output: 4
```

7. Modulo(%):

Restituisce il resto della divisione.



Python

```
1 risultato = 10 % 3 # 10 diviso 3 dà resto 1
2 print(risultato) # Output: 1
```

≡ Esempio Pratico



Python

```
1 a = 15
2 b = 4
3
4 somma = a + b          # 15 + 4 = 19
5 sottrazione = a - b    # 15 - 4 = 11
6 moltiplicazione = a * b # 15 * 4 = 60
7 divisione = a / b      # 15 / 4 = 3.75
8 div_intera = a // b    # 15 // 4 = 3 (parte intera)
9 resto = a % b          # 15 % 4 = 3
10 potenza = a ** 2      # 15 ** 2 = 225
11
12 print("Somma:", somma)
13 print("Sottrazione:", sottrazione)
14 print("Moltiplicazione:", moltiplicazione)
15 print("Divisione:", divisione)
16 print("Divisione intera:", div_intera)
17 print("Resto della divisione:", resto)
18 print("Potenza:", potenza)
```

Output:



YAML

```
1 Somma: 19
2 Sottrazione: 11
3 Moltiplicazione: 60
4 Divisione: 3.75
5 Divisione intera: 3
6 Resto della divisione: 3
```

Utilizzo degli operatori logici

Questi operatori sono usati spesso per eseguire calcoli all'interno dei programmi, come:

- **Sommare valori inseriti dall'utente.**
- **Calcolare sconti, tasse o percentuali.**
- **Gestire contatori nei loop (es. `x += 1`).**
- **Effettuare operazioni matematiche complesse, come potenze ed espressioni.**

Operatori di confronto (Comparison Operators)

Gli operatori di confronto sono utilizzati per confrontare 2 valori.

Servono per comparare i valori booleani.

Di conseguenza il risultato di un confronto è un valore booleano:

- **True (vero):**
se la condizione è soddisfatta
- **False (Falso):**
Se la condizione non è soddisfatta

Lista operatori di confronto

Operatore	Significato	Esempio (x=10, y=5)	Risultato
<code>==</code>	Uguale a	<code>x == y</code>	False
<code>!=</code>	Diverso da	<code>x != y</code>	true
<code>></code>	Maggiore di	<code>x > y</code>	true
<code><</code>	Minore di	<code>x < y</code>	False
<code>>=</code>	Maggiore o uguale a	<code>x >= y</code>	True
<code><=</code>	Minore o uguale a	<code>x <= y</code>	False

In questa tabella sono riportati tutti gli operatori di confronto con gli esempi relativi.

1. `x == y` → verifica se 10 è uguale a 5.

- in questo caso il risultato è `false` poiché 10 non è uguale a 5
2. `x != y` → verifica se 10 è diverso da 5.
 - Il risultato è `true`, poiché 10 è diverso da 5
 3. `x > y` → Verifica se **10 è maggiore di 5**.
 - Questo è **vero**, quindi il risultato è `True`.
 4. `x < y` → Verifica se **10 è minore di 5**.
 - Questo è **falso**, quindi il risultato è `False`.
 5. `x >= y` → Verifica se **10 è maggiore o uguale a 5**.
 - Questo è **vero**, quindi il risultato è `True`.
 6. `x <= y` → Verifica se **10 è minore o uguale a 5**.
 - Questo è **falso**, quindi il risultato è `False`.

Casi d'uso degli operatori di confronto

Gli operatori di confronto sono utilizzati:

- Nelle istruzioni condizionali come `if` per prendere decisioni.
- Nei loop per controllare quando terminare un ciclo.
- Per confrontare dati e variabili all'interno di calcoli e algoritmi.

☰ Esempio con `if`

```
Python
1 x = 10
2 y = 5
3
4 if x > y:
5     print("x è maggiore di y") # Questa frase viene stampata
6 else:
7     print("x non è maggiore di y")
```

Output:

```
Python
1 x è maggiore di y
```

Gli operatori logici

Gli operatori logici vengono utilizzati per combinare condizioni che restituiscono valori booleani (True o False).

In Python ci sono tre principali operatori logici:

and	restituisce vero se entrambi gli operatori sono vero
or	restituisce vero se uno degli operatori è vero
not	restituisce vero se l'operatore è falso

X	Y	X and Y	X or Y	not(X)	not(Y)
T	T	T	T	F	F
T	F	F	T	F	T
F	T	F	T	T	F
F	F	F	F	T	T

Operatori Logici.png

≡ Esempio Pratico

Prendiamo ad esempio; `x = True` e `y = False` :



Python

```
1 x = True
2 y = False
3
4 print(x and y)    # Entrambi devono essere True -> False
5 print(x or y)     # Almeno uno deve essere True -> True
6 print(not x)      # Inverte il valore di x -> False
7 print(x and not y) # x è True e not y è True -> True.
```

Spiegazione dettagliata dell'esempio

1. `x and y` → `True and False` :

- La condizione richiede che **entrambi** i valori siano `True` .
- Siccome `y` è `False` , il risultato è `False` .

2. `x or y` → `True or False` :

- La condizione richiede che **almeno uno** dei valori sia `True` .
- Siccome `x` è `True` , il risultato è `True` .

3. `not x` → `not True` :

- L'operatore `not` inverte il valore booleano di `x` .
- Siccome `x` è `True` , il risultato è `False` .

4. `x and not y` → `True and not False` :

- Prima viene calcolato `not y` , che diventa `True` (perché `y` è `False`).
- Quindi la condizione diventa `True and True` , che restituisce `True` .

≡ Esempio con condizioni pratiche >



Python

```
1  eta = 20
2  cittadino = True
3
4  # Verifica se una persona può votare
5  puo_votare = (eta >= 18) and cittadino
6  print(puo_votare) # Output: True
```

Spiegazione:

- `eta >= 18` → `True` (perché `20 >= 18`).
- `cittadino` → `True` .
- `True and True` → `True` , quindi la persona può votare.

Quando si usano gli operatori logici?

Gli operatori logici sono molto utili quando devi:

- **Combinare più condizioni in un'istruzione** `if` .
- **Controllare più casi all'interno di un programma.**
- **Negare (invertire) condizioni con l'operatore** `not` .

Gli operatori di assegnazione

Gli operatori di assegnazione vengono usati per modificare il valore di una variabile. Ad esempio:

- `x += 5` significa **aggiungere 5** al valore corrente di `x` e assegnare il risultato di nuovo a `x`.
- Lo stesso principio si applica con `--`, `*=`, `/=`, ecc.

Operator	Example	Same As
<code>=</code>	<code>x = 1</code>	<code>x = 1</code>
<code>+=</code>	<code>x += 1</code>	<code>x = x + 1</code>
<code>--</code>	<code>x -= 1</code>	<code>x = x - 1</code>
<code>*=</code>	<code>x *= 1</code>	<code>x = x * 1</code>
<code>/=</code>	<code>x /= 1</code>	<code>x = x / 1</code>
<code>//=</code>	<code>x //= 1</code>	<code>x = x // 1</code>
<code>%=</code>	<code>x %= 1</code>	<code>x = x % 1</code>
<code>**=</code>	<code>x **= 1</code>	<code>x = x ** 1</code>

Gli operatori di assegnazione .png

Partendo da questa tabella andiamo a spiegare riga per riga tutti gli esempi fatti:

1. `x = 1` :

- il valore di `x` è 1

2. `x += 1` :

cosa significa `x = x + 1`

- Sommiamo 1 al valore attuale di `x` (1)
- Il nuovo valore di `x` $\rightarrow$ 2

3. `x -= 1` :

cosa significa `x = x - 1`

- Sottraiamo 1 al valore attuale di `x` (2).
- Nuovo valore `x` $\rightarrow$ 1

4. `x *= 1 :`

cosa significa `x = x * 2`

- Moltiplichiamo il valore attuale di `x (1)` per 1.
- Nuovo valore di `x` $\rightarrow 1$

5. `x /= 1 :`

cosa significa `x = x / 1`

- Dividiamo il valore attuale di `x (1)` per 1
- Nuovo valore di `x` $\rightarrow 1.0$ (restituisce il risultato in float)

6. `x //= 1 :`

cosa significa `x = x // 1`

- Si usa la divisione intera (`//`), che restituisce solo la parte intera della divisione
- Dividiamo il valore attuale di `x (1)` per 1.
- Nuovo valore di `x` $\rightarrow 1$

7. `x %= 1 :`

Cosa significa `x = x % 1`

- L'operatore `%` calcola il resto della divisione
- Dividiamo il valore attuale di `x (1)` per 1
 - Quoziente = 1, resto = 0
- Nuovo valore di `x` $\rightarrow 0$

7. `x = 1 ** cosa` significa `x = x ** 1`

- L'operatore eleva il valore di `x` all'potenza indicata (in questo caso, 1).
- Eleviamo il valore attuale di `x (1)` alla potenza di 1
- Nuovo valore di `x` $\rightarrow 1$

Precedenza degli operatori

La precedenza degli operatori in Python si riferisce alle regole che determinano l'ordine in cui gli operatori vengono valutati in un'espressione.

Quando un'espressione contiene più operatori, Python segue un ordine di precedenza specifico per determinare quali operazioni vengono eseguite per prime.

`2 + (3 * 4) ** 2 + ((3 ** 2) / 3).`

In altre parole, segue la regola **PEMDAS**!

- Parentesi, Esponente, Moltiplica/Dividi, Aggiungi/Sottrai

Tabella delle precedenze degli operatori

()	Parentesi
**	Potenza
+x, -x, ~x	segno più (più unario), segno meno (meno unario)
*, /, //, %	Moltiplicazione, divisione, divisione del piano e modulo
+, -	Addizione e sottrazione
==, !=, >, >=, <, <=	Operatori di confronto, identità e appartenenza
not	Logical NOT
and	AND
or	OR

Esempio Pratico



Python

```
1 print(10 - 4 * 2) → 2
2 print ((10-4)* 2) → 12
```

L'associatività degli operatori

L'associatività indica l'ordine in cui vengono valutati gli operatori quando più operatori della stessa precedenza compaiono in un'espressione.

In Python:

- Quasi tutti gli operatori seguono l'associatività da sinistra a destra (left-to-right associativity).
- L'operatore esponenziale (**) è l'unico che segue l'associatività da destra a sinistra (right-to-left associativity**).

Esempi con associatività da sinistra a destra

Quando più operatori con la stessa precedenza compaiono, l'espressione viene valutata da sinistra verso destra.

Esempio 1: Moltiplicazione e Divisione



Python

```
1 print(5 * 2 // 3)
```

- Gli operatori `*` (moltiplicazione) e `//` (divisione intera) hanno **la stessa precedenza**.
- **Poiché seguono l'associatività da sinistra a destra:**

1. Calcoliamo $5 * 2 \rightarrow 10$.

2. Poi dividiamo $10 // 3 \rightarrow 3$ (divisione intera).

Output:

```
1 3
```

Esempio 2: Modifica con le parentesi



Python

```
1 print(5 * (2 // 3))
```

- **Le parentesi hanno la precedenza più alta e forzano la valutazione di `2 // 3` per primo.**
- $2 // 3 \rightarrow 0$ (divisione intera).
- Poi moltiplichiamo $5 * 0 \rightarrow 0$.

Output:

```
1 0
```

Esempi con associatività da destra a sinistra (Esponenziale)

L'operatore **`**` esponenziale** `**` segue l'associatività da destra a sinistra `**`.

Esempio 1: Senza parentesi



Python

```
1 print(2 ** 3 ** 2)
```

- Seguiamo l'associatività da **destra a sinistra**:

1. Calcoliamo $3 ** 2 \rightarrow 9$.
2. Poi calcoliamo $2 ** 9 \rightarrow 512$.

Output:

```
1 512
```

Esempio 2: Con parentesi



Python

```
1 print((2 ** 3) ** 2)
```

- Le **parentesi** forzano la valutazione di $2 ** 3$ per primo:

1. Calcoliamo $2 ** 3 \rightarrow 8$.
2. Poi calcoliamo $8 ** 2 \rightarrow 64$.

Output:

```
1 64
```

Suggerimento pratico

Quando hai dubbi su come vengono valutati gli operatori, **usa le parentesi** per rendere chiaro l'ordine delle operazioni!

Per ricapitolare

1. **Quasi tutti gli operatori** in Python seguono l'**associatività da sinistra a destra**.
- Esempi: `+`, `-`, `*`, `/`, `//`, `%`.
2. **L'operatore esponenziale `**` segue l'associatività da destra a sinistra**.

3. Per evitare confusione, usa sempre le parentesi `()` per controllare l'ordine di valutazione e migliorare la leggibilità del codice.

Altri operatori

1. Identity Operators

Gli Identity Operators servono a confrontare l'identità di due oggetti, ovvero verificano se due oggetti puntano alla stessa posizione di memoria.

Operatori:

- `is` → Restituisce `True` se due variabili fanno riferimento allo stesso oggetto.
- `is not` → Restituisce `True` se due variabili fanno riferimento a oggetti diversi.

Esempio:



Python

```
1 a = 5
2 b = a # b fa riferimento allo stesso oggetto di a
3 c = 7
4
5 print(a is b)      # True: a e b puntano allo stesso oggetto
6 print(a is c)      # False: a e c puntano a oggetti diversi
7 print(a is not c)  # True: a e c sono oggetti diversi
```

Spiegazione:

- `a` e `b` hanno lo stesso valore e fanno riferimento allo stesso oggetto in memoria → `a is b` restituisce `True`.
- `a` e `c` hanno valori diversi → `a is c` restituisce `False`.

2. Bitwise Operators

Gli operatori bitwise lavorano a livello di bit (il sistema binario). Questi operatori manipolano i bit individuali dei numeri interi.

Tabella operatori Bitwise

Operatore	Operatore 2	Significato
&	AND	Restituisce 1 se entrambi i bit sono 1
	OR	Restituisce 1 se almeno uno dei bit è 1
^	XOR	Restituisce 1 se i bit sono diversi
~	NOT	Inverte i bit (negazione a complemento a due)
<<	Left Shift	sposta i bit a sinistra, aggiungendo zeri a destra
>>	Right Shift	Sposta i bit a destra, eliminando i bit più a destra

☰ Esempi

1. AND (&):



Python

```
1 5 & 3
```

- 5 in binario: 101
- 3 in binario: 011
- AND a livello di bit → 001 (solo il primo bit è 1).
- Risultato: 1.

2. OR (|):



Python

```
1 5 | 3
```

- 5 in binario: 101
- 3 in binario: 011
- OR a livello di bit → 111 (almeno un bit è 1).
- Risultato: 7.

3. XOR (^):



Python

```
1 5 ^ 3
```

- **5 in binario:** 101
- **3 in binario:** 011
- XOR a livello di bit → 110 (bit diversi restituiscono 1).
- **Risultato:** 6 .

4. NOT (~):



Python

```
1 ~5
```

- **5 in binario:** 101 .
- Inversione dei bit (complemento a due): ...11111010 .
- In Python, questo rappresenta **-6**.

Spiegazione: In un complemento a due, invertire i bit e aggiungere 1 porta al numero negativo.

5. Left Shift (<<):



Python

```
1 5 << 2
```

- **5 in binario:** 101 .
- Spostiamo i bit **2 posizioni a sinistra**, aggiungendo zeri a destra → 10100 .
- **Risultato:** 20 .

6. Right Shift (>>):



Python

```
1 5 >> 2
```

- **5 in binario:** 101 .
- Spostiamo i bit **2 posizioni a destra**, eliminando i bit più a destra → 001 .

- Risultato: 1 .

3.List Operators

Gli operatori per le liste servono a controllare la presenza o assenza di un elemento in una lista, stringa o sequenza.

Operatori:

1. in
2. not in

Operatori	Spiegazione
in	Restituisce True se un elemento è presente nella sequenza
not in	Restituisce True se un elemento non è presente nella sequenza

Esempio:



Python

```
1 string1 = 'hello'
2
3 print('h' in string1)      # True: 'h' è nella stringa 'hello'
4 print('c' in string1)      # False: 'c' non è nella stringa
                             'hello'
5 print('c' not in string1)  # True: 'c' non è presente nella
                             stringa
```

Spiegazione:

- 'h' in string1 → 'h' è il primo carattere della stringa 'hello' , quindi restituisce True .
- 'c' in string1 → 'c' non è nella stringa, quindi restituisce False .
- 'c' not in string1 → 'c' non è presente, quindi restituisce True .

☰ Per Ricapitolare

1. **Identity Operators** (`is` , `is not`):
verificano se due variabili puntano allo stesso oggetto in memoria.
2. **Bitwise Operators**:
Manipolano i bit di un numero intero.
3. **List Operators** (`in` , `not in`):
Controllano la presenza o assenza di elementi in sequenze.

Operatori delle stringhe

Gli operatori per le stringhe funzionano in modo simile a quelli utilizzati per i numeri, ma con alcune differenze specifiche.

1. Concatenazione di stringhe (`+`)

L'operatore `+` permette di concatenare due stringhe, ossia unire le due stringhe in una sola.



Python

```
1 print("My name is " + "Alice") #output: "My name is Alice"
```

2. Ripetizione (`*`)

L'operatore `*` permette di ripetere una stringa un certo numero di volte.



Python

```
1 print("Blah " * 3) #output: Blah Blah Blah
```

3. Confronto

Gli operatori `==` e `!=` permettono di confrontare due stringhe

- `==` restituisce `True` se le due stringhe sono uguali
- `!=` restituisce `True` se le due stringhe sono diverse

Esempio:



Python

```
1 print("Alice" == "Bob") # False
2 print("Alice" != "Bob") # True
```

- `"Alice" == "Bob"` → Le due stringhe non sono uguali, quindi restituisce `False`.
- `"Alice" != "Bob"` → Le due stringhe sono diverse, quindi restituisce `True`.

Cosa succede con gli errori nelle espressioni?

Definiamo le seguenti variabili:

- `x` contenente un **intero**.
- `y` contenente **0**.
- `s` contenente una **stringa**.

Ora vediamo cosa succede con queste espressioni:

1. `x / y`

- Cosa succede?

Se proviamo a fare la divisione di `x` per `y` dove `y = 0`, otteniamo un errore perché non è possibile dividere per zero.

Questo è un errore di **ZeroDivisionError**.



Python

```
1 x = 5
2 y = 0
3 print(x / y)
```

Risultato:

```
1 ZeroDivisionError: division by zero
```

2. `s + x`

- Cosa succede?

Quando proviamo a concatenare una stringa (`s`) con un intero (`x`) usando l'operatore `+`, otteniamo un errore perché Python non può unire una stringa e un numero senza fare una conversione esplicita.

Dobbiamo prima **convertire l'intero in una stringa**.



Python

```
1 s = "Hello"
```

```
2 x = 5
3 print(s + x) # Error
```

Risultato



Python

```
1 TypeError: can only concatenate str (not "int") to str
```

- Per fare ciò dobbiamo usare `str(x)` :



Python

```
1 print(s + str(x)) # Funziona
```

Risultato:

```
1 Hello5
```

3. `s*y` :

- Cosa succede?

L'operatore `*` per le stringhe ripete la stringa un certo numero di volte. Se `y` è 0 (o la variabile a cui si assegna un integer che come valore ha 0), la stringa verrà ripetuta 0 volte, quindi il risultato sarà una stringa vuota.



Python

```
1 s = "Hello"
2 y = 0
3 print(s * y) # Risultato: ""
```

Questo torna utile se si vuole "annullare" una stringa, cioè ripeterla 0 volte

☰ Per Ricapitolare

- **Concatenazione** (`+`) unisce due stringhe.
- **Ripetizione** (`*`) ripete una stringa un certo numero di volte.
- **Confronto** (`==`, `!=`) confronta due stringhe.
- Con l'operatore `/` non possiamo dividere per zero, e con `+` non possiamo unire una stringa e un numero senza fare la conversione

esplicita.

- L'operatore `*` con `0` ripete la stringa 0 volte, producendo una stringa vuota.